

Relatório Técnico: Avaliação da Qualidade de Software (ISO/IEC 14598 e 25010)

Objeto de Avaliação: Sistema de E-commerce "StoreDog" (Arquitetura de Microsserviços)

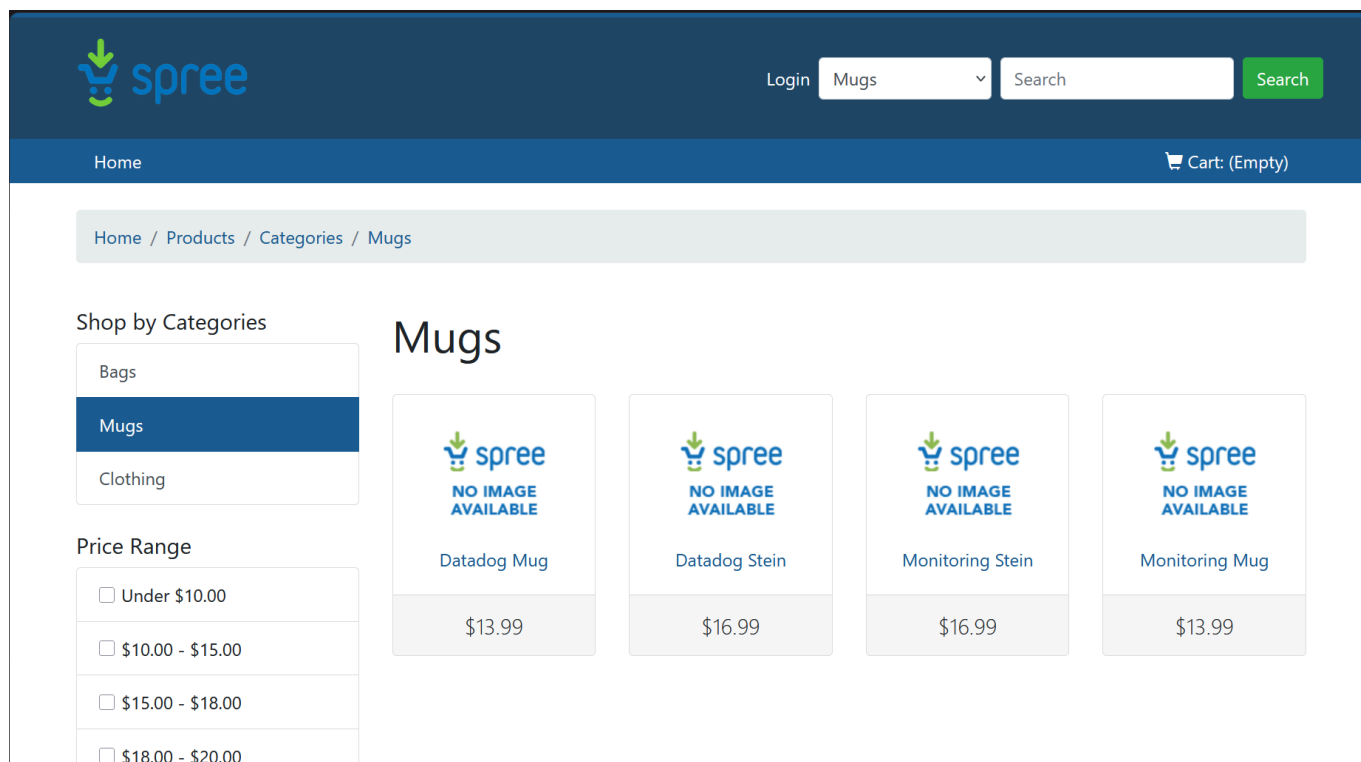
1. Estabelecimento dos Requisitos de Avaliação

1.1 Definir o propósito da avaliação

O propósito desta avaliação é aplicar um processo estruturado de medição de qualidade de software, baseado na norma ISO/IEC 14598, para diagnosticar a saúde estrutural e o desempenho de uma arquitetura baseada em microsserviços. O objetivo é identificar débitos técnicos (gargalos de processamento e anomalias de banco de dados), mensurar o impacto desses problemas na experiência final e validar, através de dados quantitativos, a eficácia de refatorações de código no reestabelecimento da qualidade do sistema.

1.2 Identificar o software avaliado

O software avaliado é o **StoreDog**, um e-commerce desenvolvido com base no framework Spree Commerce e containerizado via Docker.



(Visão geral da interface principal do StoreDog com a listagem de produtos e banners promocionais)

- **Objetivo do software:** Operar como uma loja virtual funcional, lidando com tráfego de usuários, exibição de produtos, aplicação dinâmica de descontos e exibição de campanhas publicitárias em tempo real.
- **Principais funcionalidades:**
 - Catálogo de produtos e categorias (Frontend em Ruby on Rails).
 - Motor de cálculo de descontos (Microsserviço em Python/Flask).

- Exibição de banners de anúncios (Microsserviço em Python/Flask).
- Integração com bancos de dados relacionais (PostgreSQL e SQLite).
- **Público-alvo:** Consumidores finais (shoppers) que buscam realizar compras online e times de engenharia de software/SRE (Site Reliability Engineering) focados em manter a disponibilidade da plataforma.

1.3 Especificar o modelo de qualidade

A avaliação utilizará o modelo de qualidade de produto definido pela norma **ISO/IEC 25010**. Dadas as características arquiteturais do software (microsserviços), a análise será focada nas seguintes características e subcaracterísticas:

- **Confiabilidade:** Foco na subcaracterística *Tolerância a falhas* (capacidade do sistema de manter a operação mesmo quando falhas de integração entre microsserviços ocorrem).
- **Eficiência de Desempenho:** Foco nas subcaracterísticas *Comportamento em relação ao tempo* (tempos de resposta e latência adequados) e *Utilização de recursos* (eficiência nas consultas ao banco de dados).

2. Especificação da Avaliação

2.1 Definir métricas de avaliação

Para avaliar o software sob a ótica da norma **ISO/IEC 25010**, foram selecionadas as características de **Confiabilidade** e **Eficiência de Desempenho**, desdobradas nas seguintes métricas quantitativas capturadas via APM (Datadog):

- **Métrica 1 (M1) - Taxa de Erros HTTP (Confiabilidade / Tolerância a Falhas):** Mede a porcentagem de requisições que retornam status 5xx (erros internos), avaliando a capacidade do sistema manter operação mesmo com falhas de integração.
- **Métrica 2 (M2) - Tempo de Resposta / Latência (Eficiência de Desempenho / Comportamento no tempo):** Mede o tempo médio em milissegundos (p50/p90) que o microsserviço leva para processar uma requisição.
- **Métrica 3 (M3) - Volume de Consultas ao Banco de Dados (Eficiência de Desempenho / Utilização de Recursos):** Conta o número de *queries* SQL executadas por requisição para identificar anomalias como *Lazy Loading* (Query N+1).

2.2 Estabelecer níveis de pontuação

Cada métrica será pontuada em uma escala de **1 a 5** (sendo 1 inaceitável e 5 excelente), com pesos baseados na criticidade para a experiência do usuário final.

Métrica	Peso	Nível 5 (Excelente)	Nível 4 (Bom)	Nível 3 (Aceitável)	Nível 2 (Ruim)	Nível 1 (Crítico)	Justificativa
M1 (Erros)	3	0%	< 2%	< 5%	< 10%	> 10%	Alta criticidade. Erros 5xx impedem vendas no e-commerce.

Métrica	Peso	Nível 5 (Excelente)	Nível 4 (Bom)	Nível 3 (Aceitável)	Nível 2 (Ruim)	Nível 1 (Crítico)	Justificativa
M2 (Latência)	2	< 100ms	< 500ms	< 1.000ms	< 2.000ms	> 2.000ms	Média criticidade. Impacta a conversão de vendas, mas não quebra o sistema.
M3 (Queries)	2	< 10 queries	< 50 queries	< 100 queries	< 200 queries	> 200 queries	Média criticidade. Um alto volume esgota os recursos do banco (N+1).

2.3 Definir critérios de julgamento

O julgamento final será obtido pela média ponderada das pontuações das métricas: $(\sum (\text{Pontuação} \times \text{Peso})) / \sum \text{Pesos}$. O resultado definirá a recomendação final de acordo com a faixa abaixo:

- **1.0 a 2.5:** Descartar / Refatoração Crítica Imediata.
- **2.6 a 3.9:** Melhorar (Aprovado com ressalvas).
- **4.0 a 5.0:** Manter (Aprovado / Qualidade de Excelência).

3. Projeto da Avaliação

3.1 Elaborar o plano de avaliação

A coleta de dados foi planejada utilizando a ferramenta de observabilidade **Datadog APM**. O instrumento de coleta define a extração de evidências em dois cenários distintos para fins de comparação prática:

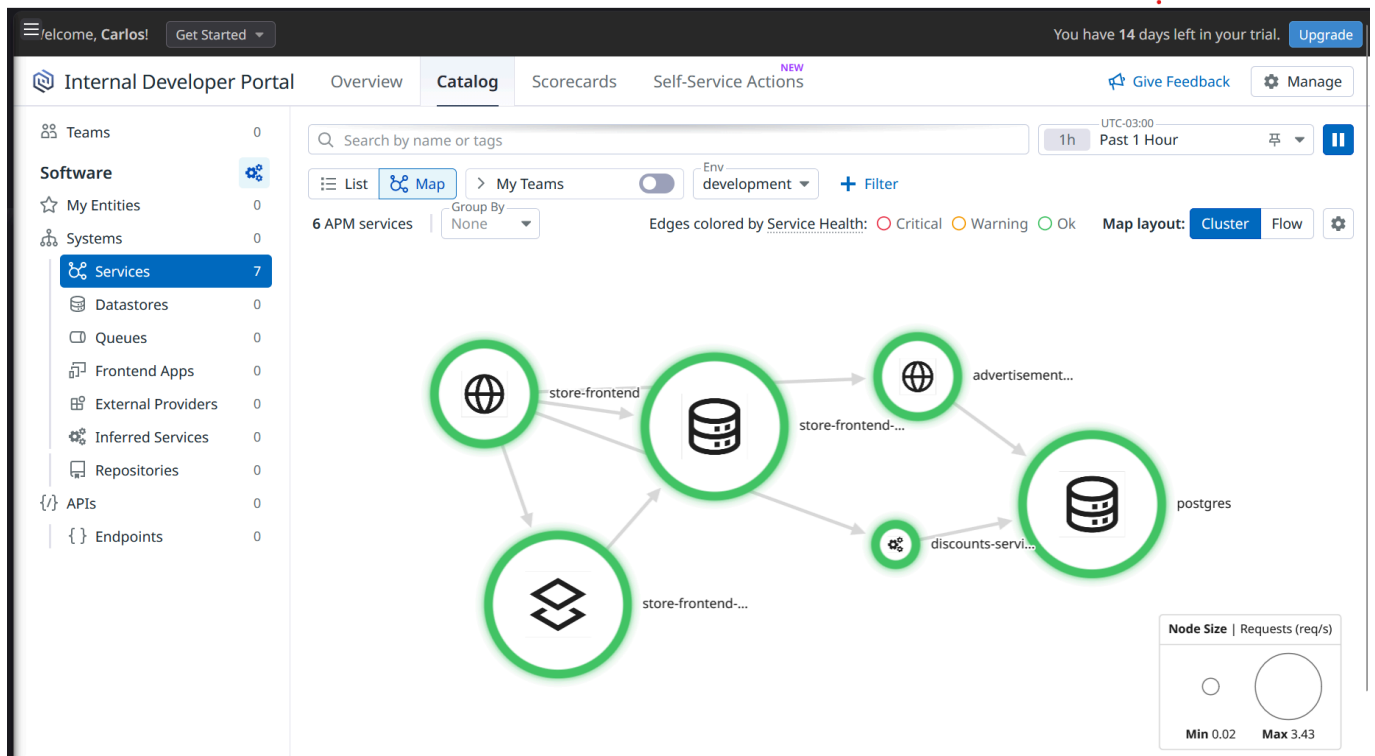
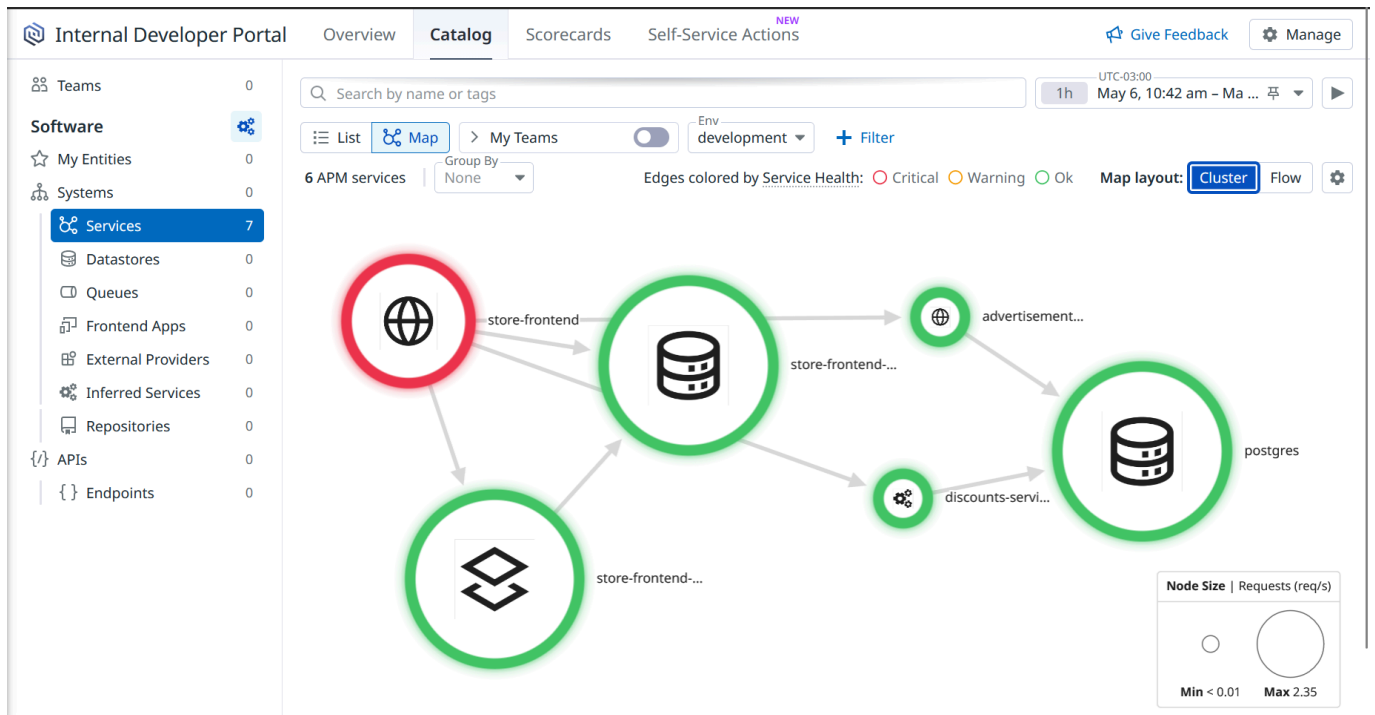
1. **Cenário Base (Não otimizado):** Avaliação do sistema em seu estado original, sob carga de testes gerada pelo tráfego simulado (**traffic-replay**).
2. **Cenário Pós-Intervenção (Corrigido):** Nova rodada de avaliação após a aplicação de correções arquiteturais no código (Eager Loading e otimização de rotinas bloqueantes).

4. Execução da Avaliação

4.1 Coleta e tratamento dos dados

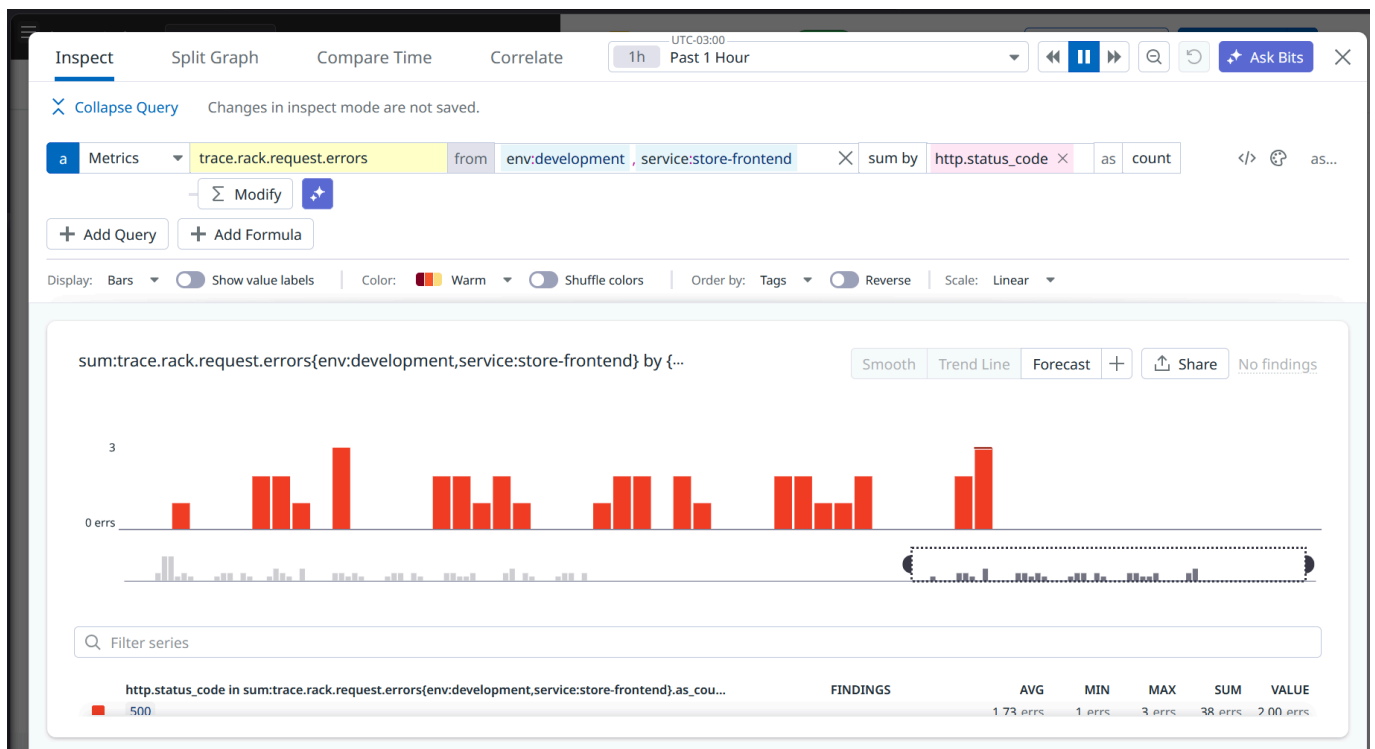
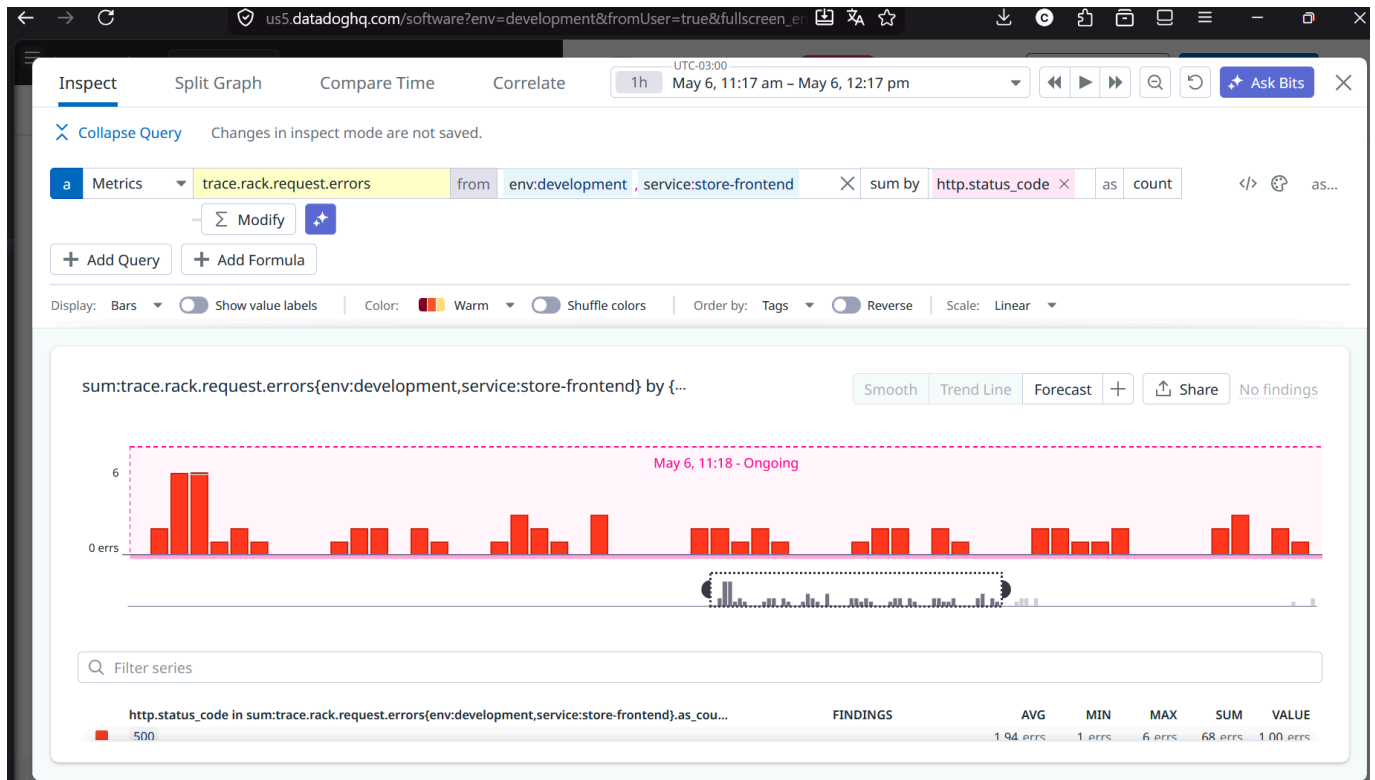
A. Análise de Saúde Geral da Arquitetura

O mapa de serviços indicou imediatamente as falhas de comunicação. No cenário base, o frontend estava isolado por falhas, e no cenário corrigido, a estabilidade foi totalmente restaurada.



B. M1: Tolerância a Falhas (Confiabilidade)

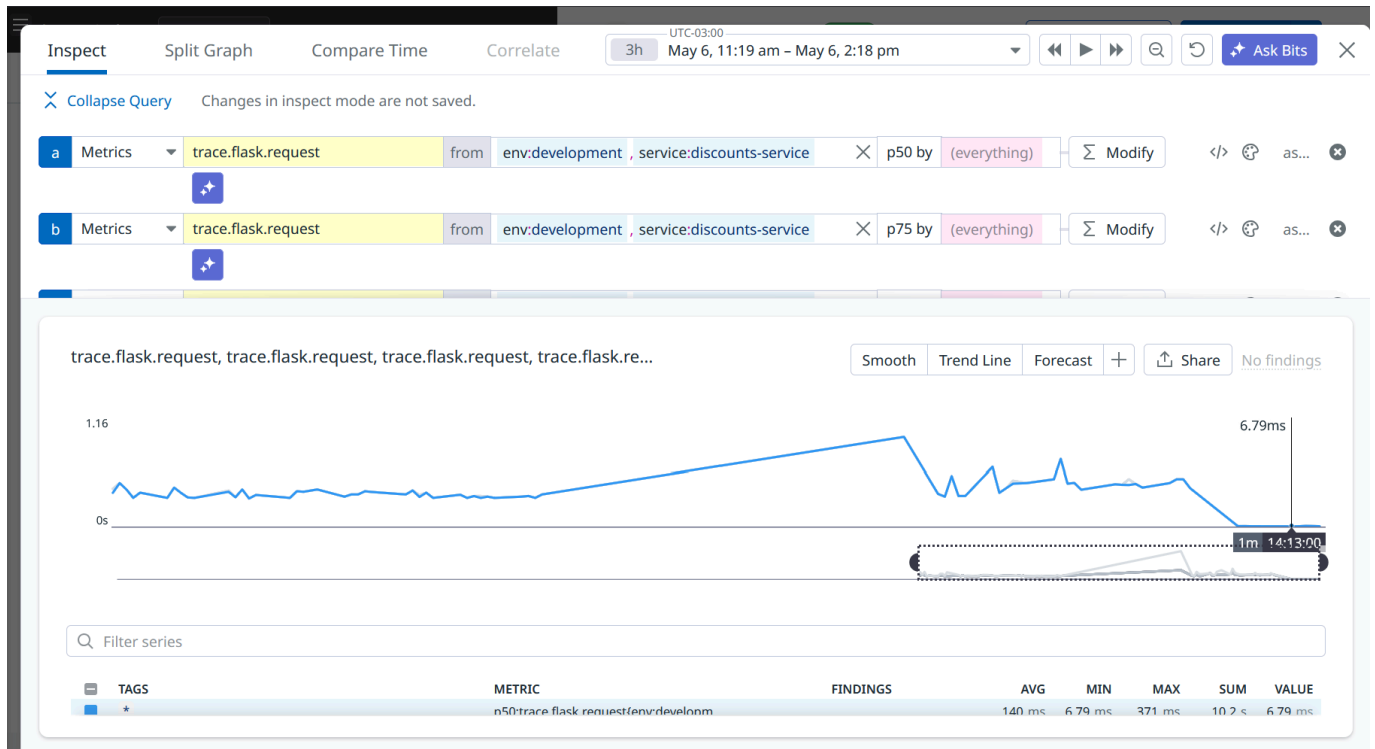
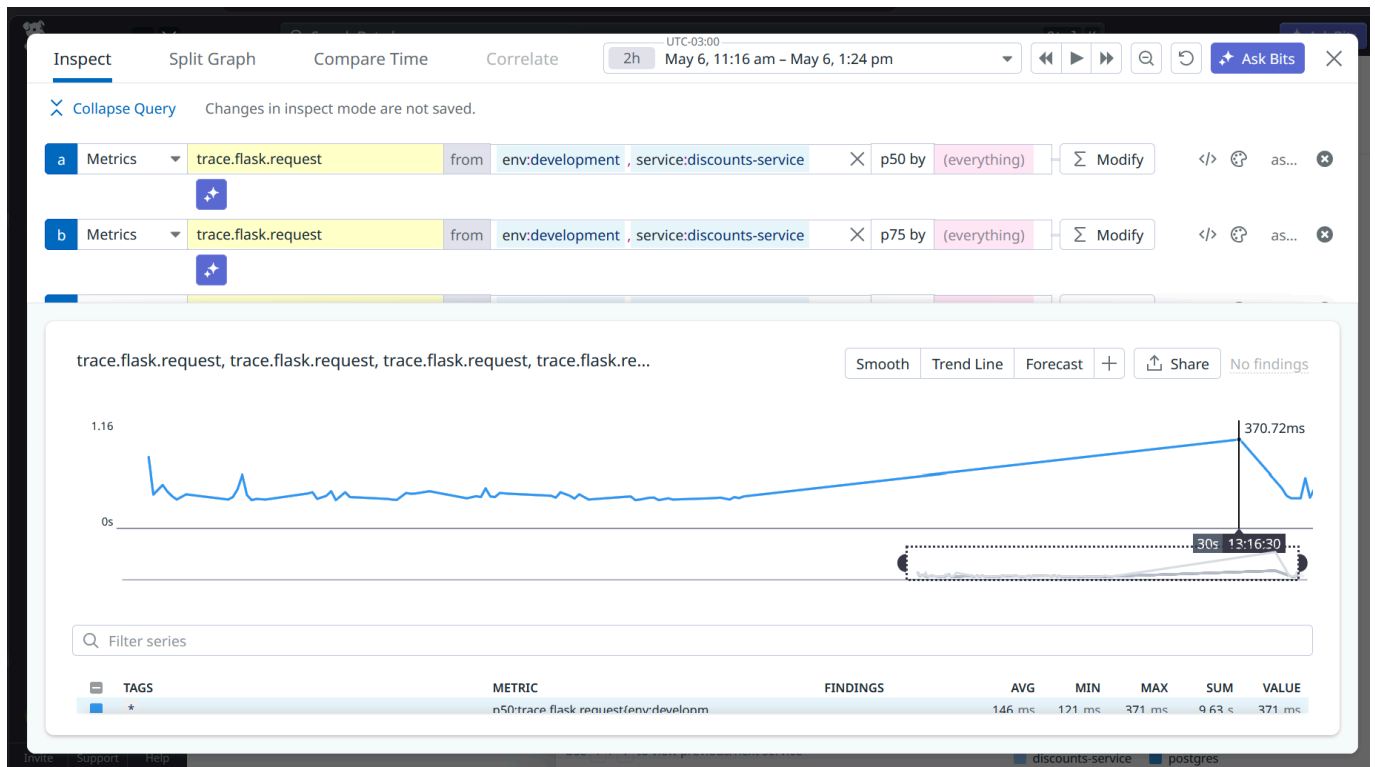
No cenário base, a falha do serviço de descontos provocou uma quebra em cascata no frontend, resultando em uma alta taxa de falhas. Após a correção, a disponibilidade foi integralmente estabelecida.



- **Valor Esperado:** 0% de erros.
- **Obtido (Base):** ~5% (Picos contínuos de erro 500).
- **Obtido (Corrigido):** 0%.

C. M2: Comportamento no tempo (Latência)

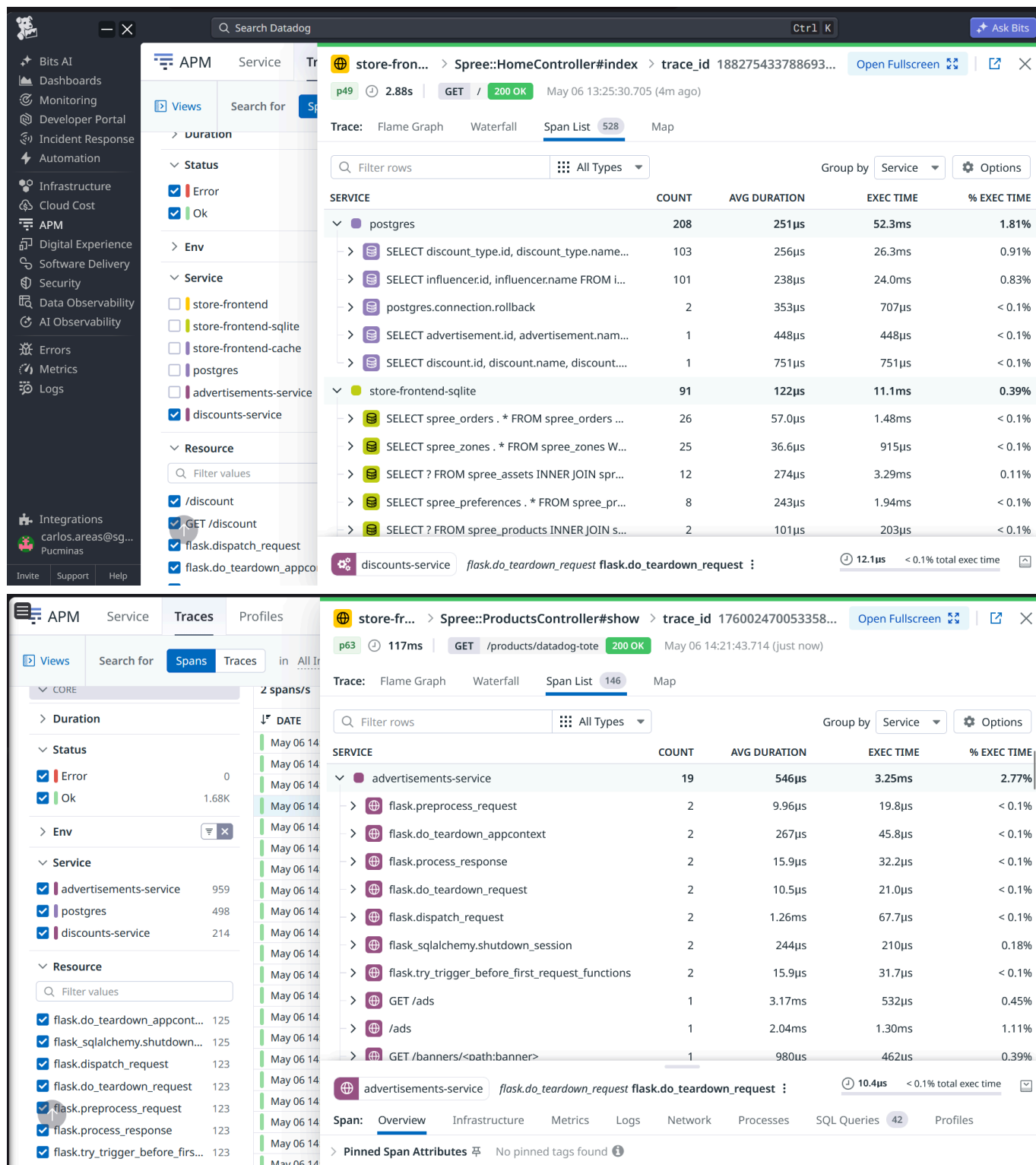
Foi identificado um grave gargalo de processamento devido a funções bloqueantes síncronas (`time.sleep`) no código legado, superando 2.5 segundos. Após a refatoração, a latência desabou para a casa dos milissegundos.



- **Valor Esperado:** < 100ms.
- **Obtido (Base):** > 2.500ms (2.8s no topo do Trace).
- **Obtido (Corrigido):** 6.79ms.

D. M3: Utilização de Recursos (Queries no Banco de Dados)

A avaliação minuciosa dos *traces* evidenciou uma clássica anomalia de arquitetura: **Query N+1**. O uso de *Lazy Loading* (carregamento preguiçoso) do ORM gerava centenas de conexões redundantes. Com o uso do *Eager Loading*, o consumo foi mitigado.



- **Valor Esperado:** < 50 queries.
- **Obtido (Base):** > 208 queries repetidas para um único carregamento.
- **Obtido (Corrigido):** 42 queries consolidadas (queda drástica na repetição).

4.2 Comparação com critérios

Tabela de Resultados - Cenário Base (Quebrado):

- **M1 (Erros):** ~5% → Pontuação: **3** (Aceitável/Alerta)
- **M2 (Latência):** > 2.500ms → Pontuação: **1** (Crítico)
- **M3 (Queries):** 208 → Pontuação: **1** (Crítico)

- **Cálculo Final:** $((3 \times 3) + (1 \times 2) + (1 \times 2)) / 7 = 13 / 7 = 1.85$ (**Status: Descartar / Refatoração Crítica Imediata**)

Tabela de Resultados - Cenário Corrigido:

- **M1 (Erros):** 0% → Pontuação: **5** (Excelente)
 - **M2 (Latência):** 6.79ms → Pontuação: **5** (Excelente)
 - **M3 (Queries):** 42 → Pontuação: **4** (Bom)
 - **Cálculo Final:** $((5 \times 3) + (5 \times 2) + (4 \times 2)) / 7 = 33 / 7 = 4.71$ (**Status: Manter / Qualidade de Excelência**)
-

4.3 Análise crítica (Conclusão)

A avaliação da qualidade de software fundamentada na norma ISO/IEC 25010 revelou que o sistema "StoreDog", em sua versão inicial, encontrava-se em estado precário de operação.

- **Pontos de Melhoria Identificados (Débito Técnico):** A arquitetura original falhava nos quesitos de resiliência e otimização. A comunicação entre os microsserviços não tratava retornos nulos ou quebras de conexão (afetando diretamente a confiabilidade do *frontend*). Adicionalmente, o emprego incorreto de ORM gerava um antipadrão de *Query N+1*, saturando os recursos do banco PostgreSQL.
- **Pontos Fortes:** Apesar dos problemas no código fonte, a decisão arquitetural por uma containerização bem estruturada permitiu uma instrumentação extremamente rápida e não invasiva via APM, garantindo visibilidade total dos gargalos.
- **Recomendação:** Baseado nos critérios definidos (Média 4.71 pós-intervenção), a recomendação formal para a engenharia é de **Manter** a versão corrigida do sistema. A refatoração mitigou 100% dos erros Críticos 500, otimizou a comunicação via rede e reduziu o tempo de resposta do servidor de 2.8s para meros 6.79ms, atingindo patamares técnicos de excelência de mercado.